

CONTENTS

- Overview
- 1 Why Long Prompts Feel Slow: The Two Phases of LLM Inference
- 2 The KV Cache: Trading Memory for Compute
- 3 How Naive KV Cache Allocation Wastes GPU Memory
- 4 PagedAttention: Virtual Memory for the GPU
- 5 Tuning vLLM for Maximum Throughput
- 6 Speculative Decoding: Spending Idle GPU Cycles
- Glossary
- Check yourself
- Where to go next

How KV Cache and PagedAttention Speed Up LLM Inference

Understand why serving an LLM to many users is a memory problem, and how KV cache, PagedAttention, and vLLM's tuning knobs turn wasted GPU memory into throughput.

For engineers who deploy or serve LLMs in production and want to understand where GPU memory and latency actually go · 27 July 2026

[Watch the source video](#)[Read the condensed version](#)[Read the highlights](#)

BEFORE YOU START

- Basic understanding of the transformer attention mechanism (queries, keys, values)
- Familiarity with how LLMs generate text one token at a time
- Basic GPU/VRAM concepts (what GPU memory is used for during inference)

Overview

When an LLM API feels fast for one user and grindingly slow for a hundred, the model itself is rarely the problem. The bottleneck is almost always how the serving system manages GPU memory during inference, specifically the context the model has to keep readily available for every request it is handling at once.

LLM inference splits into two phases with different bottlenecks: a compute-bound prefill phase that processes the whole prompt at once, and a memory-bound decode phase that generates one token at a time while repeatedly reading back everything generated so far. The KV cache exists to make decode cheaper by avoiding redundant recomputation, but naive implementations of that cache waste most of the memory they reserve. PagedAttention, from the open-source vLLM project, fixes that waste by applying the same virtual-memory paging trick operating systems use for RAM.

This lesson walks through both phases, explains why the KV cache is a memory-for-compute trade, shows exactly how naive allocation wastes 60-80% of that memory, and covers the practical vLLM settings — GPU memory utilization, prefix caching, chunked prefill, and speculative decoding — that convert PagedAttention's efficiency into real throughput and latency gains you can configure today.

KEY TAKEAWAYS

- ✓ LLM inference has two distinct phases: prefill (compute-bound, processes the whole prompt at once) and decode (memory-bound, generates one token at a time).
- ✓ The KV cache avoids recomputing attention keys and values for every prior token at each generation step, trading GPU memory for compute.
- ✓ Model weights alone can consume the majority of a GPU's memory, leaving a small, fixed budget that every concurrent request's KV cache has to share.
- ✓ Naive fixed-block KV cache allocation reserves memory for the worst-case sequence length per request, and research cited in the video puts the resulting waste at 60-80% of the KV cache budget.
- ✓ PagedAttention applies OS-style virtual memory paging — small fixed-size, non-contiguous pages plus a block table — to eliminate internal fragmentation, external fragmentation, and duplicate storage of shared prefixes.
- ✓ Prefix caching and chunked prefill are practical vLLM settings that reuse KV cache across requests sharing a prompt and stop long prefills from stalling in-flight decode work, respectively.
- ✓ Speculative decoding spends otherwise-idle GPU compute during decode to draft and verify multiple tokens per step, cutting latency without changing output quality, and is most useful at low-to-moderate concurrency.

01 Why Long Prompts Feel Slow: The Two Phases of LLM Inference

0:00

Serving an LLM exposes two different bottlenecks: a compute-bound prefill phase before the first token, and a memory-bound decode phase for every token after that.

When you send a prompt to an LLM API, there is a pause before the first token appears, then tokens stream back quickly. That pause is the prefill phase: the model runs every token of your input through every transformer layer at once to build the internal representation, the attention keys and values, that it needs before it can generate anything. Prefill is compute-bound: the GPU's matrix-multiply units are the limiting factor, and the time it takes scales with prompt length and model size, not with how many other users happen to be active.

Once prefill finishes, the model enters the decode phase: it generates one output token at a time, and after each token it must attend over every token that came before it, the original prompt plus everything generated so far. Because decode only computes one new token's worth of math per step, the GPU's compute units are mostly idle during this phase; the real bottleneck is how fast the GPU can read the stored context back out of memory. That is why decode is memory-bound rather than compute-bound.

This split matters because it changes what "more users" actually costs. Adding concurrent users mostly adds parallel compute work during prefill, which GPUs handle well. But it also multiplies the amount of context that has to sit resident in GPU memory simultaneously, one growing context per active request,

during decode. When that memory becomes fragmented, full, or has to be recomputed, decode latency is what degrades first, visible as slower token streaming rather than a slower initial load.

KEY POINTS

- Prefill processes the entire input prompt in one pass and is limited by GPU compute throughput.
- Decode generates one token at a time and is limited by GPU memory bandwidth, not raw compute.
- Time-to-first-token is dominated by prefill; the cadence of streaming tokens afterward is dominated by decode.
- More concurrent users mainly stress decode, because each user's growing context must stay resident in GPU memory at the same time as everyone else's.

Spotting the two phases from the outside

Call an LLM API with a 2,000-token prompt and time it: the delay before the first streamed token is prefill; the token-by-token cadence after that is decode. Doubling the prompt length roughly doubles prefill time, but barely changes the per-token decode rate, because decode work depends on how much context must be read at each step, which grows far more slowly per step than the one-time cost of encoding the whole prompt.

02 The KV Cache: Trading Memory for Compute

2:53

Attention needs a query, key, and value for every token at every layer; caching the keys and values means each new token only computes its own instead of recomputing history.

At every transformer layer, each token produces three vectors: a query (what am I looking for), a key (what do I contain, for others to match against), and a value (what I actually contribute if matched). Attention for a given token compares its query against the keys of all preceding tokens to get relevance weights, then combines their values using those weights.

Autoregressive generation means the model produces its response one token at a time, and attention is causal: each token can only attend to itself and everything before it, never to future tokens. Naively, generating token 1000 would require recomputing the keys and values for tokens 1 through 999 all over again, even though those vectors never change once computed. Repeating that recomputation at every one of n generation steps makes total attention work grow quadratically with response length.

The KV cache removes that redundancy: once a token's key and value vectors are computed, they are stored in GPU memory. Each new token only computes its own key, value, and query, then attends over the cached key/value history of everything before it. This turns a quadratic recomputation problem into a linear

one, at the cost of memory instead of compute. That trade only pays off because spending GPU memory is far cheaper than repeating a full forward pass through every transformer layer for every prior token, on every step.

KEY POINTS

- Every token produces a query, key, and value vector at each transformer layer.
- Causal attention means token N only attends to tokens 1..N, never to tokens after it.
- Without caching, generating token N re-derives the keys and values of all N-1 prior tokens, at every step.
- The KV cache stores keys and values once computed, so later tokens only compute their own.
- This converts per-response attention compute from quadratic to linear, at the cost of GPU memory instead.

Naive recomputation versus a KV cache

Pseudocode contrasting a naive decode loop, which reruns attention over the full prefix at every step, against a cached loop, which only computes the new token's K/V/Q and appends to a running cache. The naive version does $O(n)$ work per step for n steps ($O(n^2)$ total); the cached version does $O(1)$ new work per step plus a cheap read of the cache.

```
# Naive: recompute keys/values for every prior token at every step
def generate_naive(tokens, model):
    output = []
    for i in range(len(tokens), MAX_LEN):
        # recomputes K, V for ALL tokens seen so far, every single step
        keys, values = model.compute_kv(tokens)
        query = model.compute_query(tokens[-1])
        next_token = model.attend_and_predict(query, keys, values)
        tokens.append(next_token)
        output.append(next_token)
    return output

# With a KV cache: only compute K/V for the newest token
def generate_cached(tokens, model):
    output = []
    kv_cache = model.compute_kv(tokens) # prefill: build cache once
    for i in range(len(tokens), MAX_LEN):
        query = model.compute_query(tokens[-1])
        # only this token's K, V are computed and appended
        new_k, new_v = model.compute_kv([tokens[-1]])
        kv_cache.append(new_k, new_v)
        next_token = model.attend_and_predict(query, kv_cache.keys, kv_cache.values)
        tokens.append(next_token)
        output.append(next_token)
    return output
```

03 How Naive KV Cache Allocation Wastes GPU Memory

4:20

Weights alone can eat most of a GPU's memory, and traditional serving systems reserve a fixed worst-case block of the remainder for every request, wasting most of it.

Model weights already claim a large share of GPU memory before a single request arrives. A 13-billion-parameter model like Llama 13B needs about 26 GB just to hold its weights on a 40 GB A100 card, roughly 65% of the card, because each parameter typically needs 2 bytes in FP16. That leaves only about 35% (around 14 GB) for everything else, most importantly the KV cache shared across every concurrently active request.

Traditional serving systems allocate that remaining memory naively: they pre-allocate a fixed, contiguous block per request sized to the maximum possible sequence length, say 2,048 tokens, regardless of how long the request actually turns out to be. If the average user sends 200 tokens and gets 300 back (500 tokens total), but the system reserved room for 2,048, then 1,500 tokens' worth of memory sits empty and unusable by any other request for the entire lifetime of that request.

The video cites research showing this pattern wastes 60-80% of the KV cache memory budget. That is memory that could have served additional concurrent requests but instead sits idle, capping how many users a single GPU can actually serve, which is exactly the pattern of GPU memory spiking and throughput tanking as concurrency rises, even though most of that memory is not doing useful work.

KEY POINTS

- Model weights alone can consume the majority of GPU memory (about 65% for a 13B model on a 40 GB GPU).
- Naive serving reserves a fixed block per request sized to the maximum possible output length, not actual usage.
- Reserved-but-unused memory cannot be lent to other concurrent requests while it sits idle.
- Reported waste from this fixed-block approach is 60-80% of the KV cache memory budget.

Computing reserved-but-wasted memory for one request

A simple calculation showing how much of a per-request KV cache reservation goes unused when the system reserves for the maximum context length instead of the actual token count.

```
max_context_tokens = 2048
actual_prompt_tokens = 200
actual_output_tokens = 300
actual_used_tokens = actual_prompt_tokens + actual_output_tokens # 500

reserved_but_unused = max_context_tokens - actual_used_tokens # 1548 tokens
waste_fraction = reserved_but_unused / max_context_tokens

print(f"Unused tokens reserved: {reserved_but_unused}") # 1548
print(f"Fraction wasted: {waste_fraction:.0%}") # 76%
# This single-request estimate lands inside the 60-80% waste range
# reported for naive fixed-block KV cache allocation.
```

PagedAttention borrows the OS trick of paging: it splits KV cache into small fixed-size blocks that can live anywhere in memory, addressed through a lightweight lookup table.

Naive KV cache allocation wastes memory in three specific ways. Internal fragmentation reserves an entire hotel floor, the maximum sequence length, for a guest who only needs one room. External fragmentation happens when variable-length allocations leave gaps between them: a new request needing 500 tokens' worth of memory might find enough total free memory scattered around, but no single contiguous region big enough to hold it. Redundant duplication happens when an identical system prompt is stored separately in every concurrent request's KV cache instead of once.

PagedAttention fixes all three at once by applying the same insight operating systems use for RAM: virtual memory paging. Instead of one contiguous block per request, KV cache is split into small, fixed-size pages, 16 tokens each by default. Pages are allocated on demand as generation proceeds and can live anywhere in physical GPU memory, non-contiguous. A lightweight block table, the GPU equivalent of an OS page table, maps each request's logical sequence of pages, what the model sees, to wherever those pages actually sit in VRAM.

This design removes internal fragmentation, because a request only holds as many pages as it currently needs rather than the theoretical maximum. It removes external fragmentation, because free pages of the same fixed small size can be reused anywhere, with no contiguous run required. And it enables prefix caching (covered next), because the block table can point more than one request's logical sequence at the very same physical page instead of duplicating it.

KEY POINTS

- Internal fragmentation: reserving max-length memory for a request that ends up using far less of it.
- External fragmentation: enough total free memory exists, but no single contiguous region is large enough for a new allocation.
- Redundant duplication: an identical shared prefix, such as a system prompt, gets stored separately for every request that uses it.
- PagedAttention splits KV cache into fixed-size pages, 16 tokens by default, allocated on demand rather than reserved up front.
- A block table maps logical page addresses (what the model sees) to physical addresses (where data actually lives in VRAM), the same indirection an OS uses for virtual memory.

A block table in miniature

A minimal simulation of how a block table decouples the logical page order a request sees from the physical pages actually holding data in VRAM, and how two requests can share a physical page for an identical prefix.

```
# physical_pages holds the actual KV data, indexed by physical page id
physical_pages = {} # {physical_id: kv_data_for_16_tokens}

# each request has a block table: logical page index -> physical page id
request_a_block_table = [0, 1, 2] # request A's tokens live in physical pages 0,1,2
request_b_block_table = [0, 3]    # request B shares page 0 (same system prompt!)

def read_token_kv(block_table, logical_page, offset_in_page):
    physical_id = block_table[logical_page]
    return physical_pages[physical_id][offset_in_page]

# Because both block tables point logical page 0 at physical page 0,
# the shared system prompt's KV data is stored once, not twice --
# this is exactly what prefix caching relies on.
```

05 Tuning vLLM for Maximum Throughput

7:56

Three practical vLLM settings, GPU memory utilization, prefix caching, and chunked prefill, turn PagedAttention's efficiency into measurable throughput and latency gains.

GPU memory utilization controls what fraction of the VRAM remaining after model weights is handed to KV cache pages. The default is 0.9. Push it toward 0.95 on stable, predictable workloads to pack in more concurrent requests; pull it back toward 0.8 if out-of-memory errors show up during bursty load, since spiky traffic needs headroom the default does not leave. Benchmark per model with a load-testing tool, the video points to GuideLLM, an open-source part of the vLLM project, rather than guessing at the right value.

Prefix caching builds on the fact that PagedAttention already hashes each KV page by the token sequence it contains. Enabling it means requests sharing an identical prefix, a system prompt, a few-shot preamble, the start of a multi-turn conversation, point their block table at the same physical pages instead of each recomputing and storing that prefix separately; vLLM computes it once. In RAG pipelines, multi-turn chat, and coding agents, all of which repeat a large shared prefix across many requests, hit rates of 75-95% are common, and time-to-first-token drops sharply because prefill for the shared portion is skipped entirely.

Chunked prefill addresses a different stall: by default vLLM runs a prefill to completion before resuming any decode work, which freezes already-streaming tokens for other users whenever a new long prompt arrives, visible as stutter. Chunked prefill instead prioritizes in-flight decode requests each step, then fills whatever compute budget remains with a chunk of prefill work rather than blocking on it. Production

deployments cited in the video saw roughly 50% throughput improvement from this alone. It is typically paired with raising max-num-batched-tokens above 2048, so each step has enough budget to actually admit prefill chunks alongside decode.

KEY POINTS

- `--gpu-memory-utilization` sets the fraction of post-weights VRAM given to KV cache (default 0.9, roughly 0.8-0.95 depending on how bursty traffic is).
- Benchmark per model with a load-testing tool such as GuideLLM rather than assuming the default is optimal for your workload.
- Prefix caching reuses KV pages across requests that share an identical prefix, avoiding recomputing and re-storing it.
- Shared-prefix workloads (RAG, multi-turn chat, coding agents) commonly see 75-95% prefix cache hit rates.
- Chunked prefill interleaves prefill work with ongoing decode instead of blocking decode on it, reducing stutter and improving throughput (about 50% in production reports cited).

Launching vLLM with tuned settings

A representative vLLM server launch combining the three settings discussed: a slightly raised GPU memory utilization for a stable workload, prefix caching enabled for a workload with shared prompts, and chunked prefill enabled with a raised batched-token budget.

```
vllm serve meta-llama/Llama-2-13b-chat-hf \  
  --gpu-memory-utilization 0.95 \  
  --enable-prefix-caching \  
  --enable-chunked-prefill \  
  --max-num-batched-tokens 4096
```

06 Speculative Decoding: Spending Idle GPU Cycles

9:24

During decode the GPU has spare compute sitting idle between memory reads; speculative decoding uses that slack to draft several tokens at once and verify them in a single pass.

Because decode is memory-bound, the GPU's compute units are not fully busy between the memory reads needed to fetch cached KV data; there is idle compute capacity at every step. Speculative decoding puts that idle capacity to work: a small, fast draft model proposes a short sequence of candidate next tokens ahead of time.

The large model then checks all of those candidates in a single forward pass, cheap, because verification is compute-bound and compute is exactly what is sitting idle, rather than running one sequential forward pass per token. Correct guesses are accepted and emitted immediately; the first incorrect guess is rejected and corrected by the large model, and generation continues from there. Because every accepted token is still exactly what the large model would have produced on its own, output quality is mathematically identical to running the large model alone. This is purely a latency optimization, not a quality trade-off.

The gain is largest at low-to-moderate concurrency, where the GPU has slack to spend on drafting and verifying extra tokens. At very high concurrency, the batch of concurrent requests already keeps the GPU's compute busy, so there is less idle capacity left to spend on speculation, and the benefit shrinks. That makes speculative decoding a tool for interactive, latency-sensitive workloads rather than for maximizing raw aggregate throughput at scale. vLLM ships a zero-cost n-gram speculator out of the box, well suited to structured or repetitive outputs where recent context is a strong predictor of what comes next.

KEY POINTS

- Decode leaves GPU compute partially idle because it is bottlenecked on memory reads, not math.
- A small draft model proposes multiple candidate tokens; the large model verifies them all in a single forward pass.
- Accepted tokens are exactly what the large model would have generated alone, so output quality is unchanged.
- Gains shrink at high concurrency, since a full batch already keeps the GPU's compute busy.
- vLLM includes a built-in, zero-cost n-gram speculator suited to structured or repetitive output.

Enabling vLLM's built-in n-gram speculator

The zero-cost speculative decoding option mentioned in the video: an n-gram-based draft model that needs no separate model weights, useful for latency-sensitive, structured or repetitive output tasks such as JSON extraction or code completion.

```
vllm serve meta-llama/Llama-2-13b-chat-hf \  
  --speculative-model [ngram] \  
  --num-speculative-tokens 5 \  
  --ngram-prompt-lookup-max 4
```

Glossary

Prefill phase

The compute-bound stage of LLM inference where the model processes the entire input prompt at once to build its internal representation before generating any output token.

Decode phase

The memory-bound stage of LLM inference where the model generates one output token at a time, reading the accumulated context from GPU memory at each step.

KV cache

Stored attention key and value vectors from previously processed tokens, kept in GPU memory so later tokens do not have to recompute them.

Autoregressive generation

Generating text one token at a time, where each new token is produced based on all tokens generated so far.

Causal attention

An attention pattern where a token can only attend to itself and the tokens before it, never to tokens that come later.

PagedAttention

A memory management technique for KV cache that splits it into small fixed-size, non-contiguous pages addressed through a block table, modeled on OS virtual memory paging.

Block table

A lookup structure that maps a request's logical sequence of KV cache pages to the physical pages that actually hold that data in GPU memory.

Internal fragmentation

Wasted memory that occurs when a fixed allocation is sized for the worst case but the actual usage is much smaller, leaving the unused portion locked up and unusable by others.

External fragmentation

Wasted memory that occurs when enough total free space exists but it is scattered in gaps too small individually to satisfy a new contiguous allocation.

Prefix caching

Reusing the same stored KV cache pages across multiple requests that share an identical prompt prefix, instead of recomputing and storing it separately for each.

Chunked prefill

A scheduling strategy that interleaves pieces of a new request's prefill work with ongoing decode steps, instead of finishing prefill entirely before resuming decode.

Speculative decoding

A latency optimization where a small draft model proposes several candidate tokens that a larger model verifies in one forward pass, accepting correct guesses and correcting the first wrong one.

Time to first token (TTFT)

The delay between sending a prompt and receiving the first generated token back, dominated by the prefill phase.

vLLM

An open-source LLM inference and serving engine that introduced PagedAttention and provides the tuning options for KV cache management discussed in this lesson.

Check yourself

Answer first, then open each one.

Why is prefill compute-bound while decode is memory-bound, even though both are steps of the same model doing inference?

Prefill processes every token of the prompt through every layer in one pass, so the GPU's matrix-multiply units are saturated and are the limiting factor. Decode processes only one new token per step, so the amount of new computation is small; the limiting factor becomes how quickly the GPU can read the already-computed context (the KV cache) back out of memory for each step.

Without a KV cache, why does generating the 1000th token of a response cost far more work than generating the 10th token, and how does the cache remove that cost?

Without caching, every generation step reruns attention over the full prefix seen so far, so the 1000th token requires recomputing keys and values for all 999 prior tokens, while the 10th token only recomputes 9. That makes total work grow quadratically with response length. The KV cache stores each token's key and value vectors the first time they are computed, so every later step only computes the new token's own key, value, and query and reads the rest from the cache, making the work per step constant instead of growing.

Why does reserving GPU memory for the maximum possible sequence length waste memory even when most requests are far shorter, and why can't that reserved memory just be used by other requests in the meantime?

Naive systems allocate one fixed, contiguous block per request sized to the worst case (e.g. 2048 tokens) at the moment the request starts, regardless of how many tokens it will actually use. Because the block is reserved and contiguous, the unused portion is locked to that request for its entire lifetime and cannot be handed to any other request, even though the memory sits empty; this is internal fragmentation, and research cited in the video puts the waste at 60-80% of the KV cache budget.

How does PagedAttention's block table let two different requests share an identical system prompt's KV cache without storing it twice?

PagedAttention breaks KV cache into small fixed-size pages that can live anywhere in physical memory, and each request has its own block table mapping its logical page order to physical page locations. Because the pages holding the shared prompt are hashed by their token content, two requests with the same prefix can have their block tables point their corresponding logical pages at the very same physical pages, so the data is stored once and read by both, instead of being duplicated per request.

Speculative decoding uses a smaller, less capable draft model to guess upcoming tokens. Why doesn't this reduce the quality of the final output?

The draft model's guesses are never emitted directly; the large model verifies every candidate token in a single forward pass, and only tokens the large model itself confirms are accepted. Any guess the large model would not have produced is rejected and replaced with the large model's own correct token. Since every emitted token is one the large model actually endorsed, the output is mathematically identical to what the large model would have generated running alone; the draft model only changes how fast that same output is reached, using GPU compute that would otherwise sit idle during decode.

Where to go next

- Run `vllm serve` locally with `--gpu-memory-utilization` at 0.8 versus 0.95 under a bursty load test and observe the throughput-versus-OOM tradeoff directly.
- Read the original PagedAttention paper, "Efficient Memory Management for Large Language Model Serving with PagedAttention," for the full block table and scheduling design.
- Enable `--enable-prefix-caching` on a RAG or multi-turn chat workload and measure the change in time-to-first-token and cache hit rate.
- Benchmark a model with GuideLLM before and after enabling `--enable-chunked-prefill` to see the effect on streaming stutter under concurrent load.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.